

Code-Layout, Readability and Reusability

Date	13 July 2026
Team ID	N/A
Project Name	FinRelief AI – AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S. No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8 (Python) and Prettier (React) formatting applied
2	Proper File Structure	Files and folders are logically organized	Yes	Separate folders for routes, models, services, and components
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g., emi_ratio, settlement_percentage, debt_stress_level
4	Function / Method Names	Functions are descriptively named	Yes	e.g., calculate_financial_health(), generate_negotiation_letter()
5	Code Comments	Inline and block comments explain logic	Partial	Core financial and AI logic commented; minor UI components pending
6	Modular Design	Code is split into reusable functions/modules	Yes	Financial engine, negotiation service, and auth kept as separate modules
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared utilities extracted to avoid duplication
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try/except blocks around Gemini API calls with rule-based fallback

Reusable Components / Modules:

S. No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	FinancialHealthCalculator	Python (FastAPI service)	Dashboard & Settlement Recommendation modules	High
2	NegotiationLetterService	Python (Gemini API wrapper)	AI Negotiation page and history log	High
3	LoanCard Component	React.js	Dashboard and Loan Management pages	Medium
4	AuthMiddleware	Python (FastAPI)	All protected API routes	High
5	APIClient Utility	JavaScript (Axios wrapper)	All frontend pages calling the backend	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Consistent structure across backend and frontend modules
Readability	4	Clear naming conventions; minor gaps in UI comments
Reusability	5	Core financial and AI services are cleanly decoupled and reusable
Documentation / Comments	3	Backend well documented; frontend components need more inline notes
Overall Score	4	Solid, maintainable codebase with a few areas for improvement